Visual Programming

C# (C sharp)

Introduction

Lecturer: Dr. Khaled Mohammed BinAbdl

Department of Information Technology,

College of Computers, Seiyun University

Introduction

- C# (C Sharp) is a general purpose, multi-paradigm programming language developed by Microsoft that runs on the .NET Framework.
- C# is widely used for building mobile applications, games and windows applications.
- All .NET languages compile to a common byte code (MSIL) making their integration into programs written in different languages easier.

History

- Developed by Anders Hejlsberg
- Released in 2001-2002
- According to Anders Hejlsberg, Microsoft’s Chief Architect, C# is a derivation of C++, C, Java, Modula 2, and Smalltalk
- C# combines the best features of these languages and eradicates some of their weaknesses.

Why Choose C#?

- designed from scratch with the .net framework in mind
- combines the power of C and C++ with the productivity of Visual Basic
- With its familiar syntax, the transition for Java and C++ programmers will be an easy one
- sync with current web standards and is easily integrated with existing applications.
- In today’s society where internet programming is inevitable having a language that already supports this makes the job of the developer easier.

Example of Code

The code looks a lot like Java

```
public class Example
{
    public static void Main(string[] args)
    {
        foreach (string s in args)
        {
            System.Console.WriteLine(s);
        }
    }
}
```

C# Features

1. OOP
2. Enumerators
3. Operator Overloading
4. Windows API Invocation
5. Structured Error Handling
6. Delegates
7. Namespaces

1. OOP

C# is object oriented. Every class is a subclass of an object. Everything is an object, yes even primitives. This makes generic programming easier.

Example:

```
int n = 3;
string s = n.ToString();
```

2. Enumerators

Enumerators are a borrowed idea from C/C++. This is a data type consisting of a set of of named integers.

Example:

```
enum Weekday {Mon, Tues, Wed, Thu,
                    Fri, Sat, Sun};
```

3. Operator Overloading

Operator Overloading is yet another idea borrowed from c++. This makes polymorphism easier with custom data types.

Example:

```
Currency a, b, c;
c = a + b;
```

4. Windows API Invocation

C# was built with Windows in mind. It was created to allow programmers to create Windows application easily through a wraparound API. Some other technologies supported are COM, COM+.

5. Structured Error Handling

C# introduces new error handling techniques.

- Try-catch blocks are used but with more functionality.
- It could be any of the following;
  - *try{ } catch(SomeException){ }*
  - *try{ } catch(){ } //catches any kind of exception*
  - *try{ } catch(){ } finally{ } //finally is always executed*
  - *try{ } finally{ } //finally is always executed*

- To throw an object, it has to be a subclass of System.Exception.

6. Delegates

Delegates provide a template for a single method.

Example:

```
public delegate int ArithOp(int a, int b);
...
public int DoOp(ArithOp ar)
{ return ar(a, b); }
```

7. Namespace

- Namespace is a method of organizing similar files together. This is similar in some way to the java package idea. Every program is either explicitly within a namespace or in by default.

Example1:

- ***namespace Project{ public class P1{} }***
- ***public class P2{}***

- To use a namespace, you just simply import by using the keyword using.

Example2:

using system;

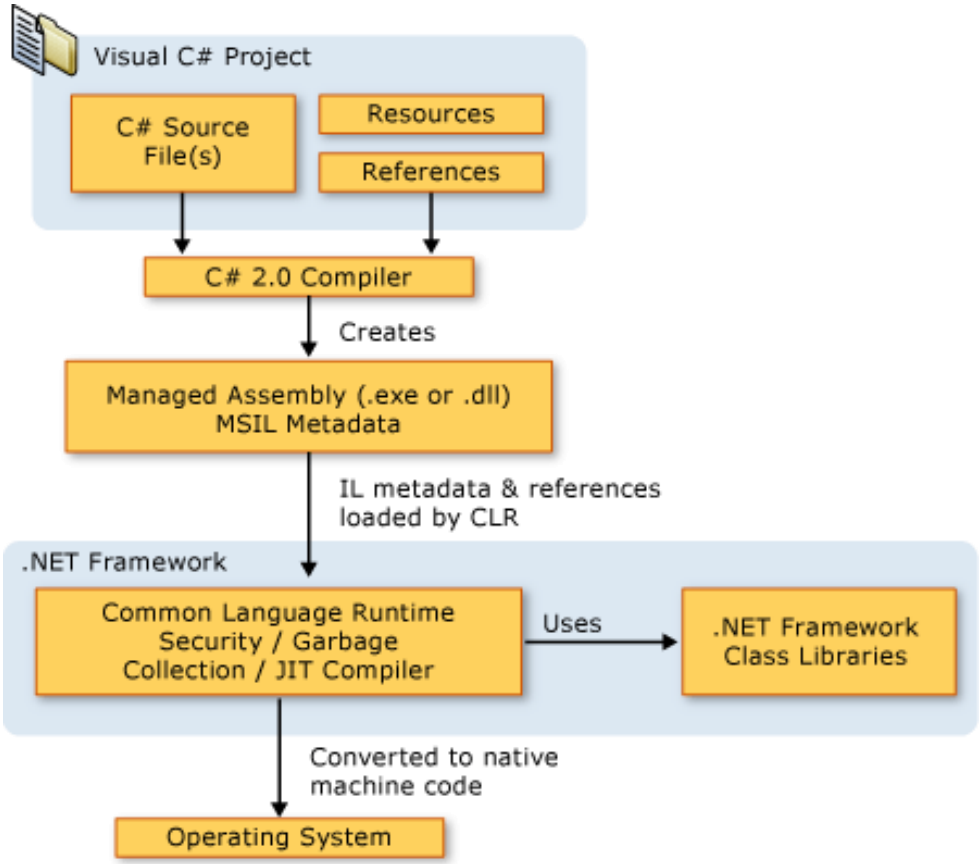
public class P1{}

Applications

- C# programs run on the .NET Framework which is an integral component of Windows that includes a virtual execution system called common language runtime (CLR).
- Likewise C# programs run on a unified set of class libraries as well.

CLR and JIT compiling

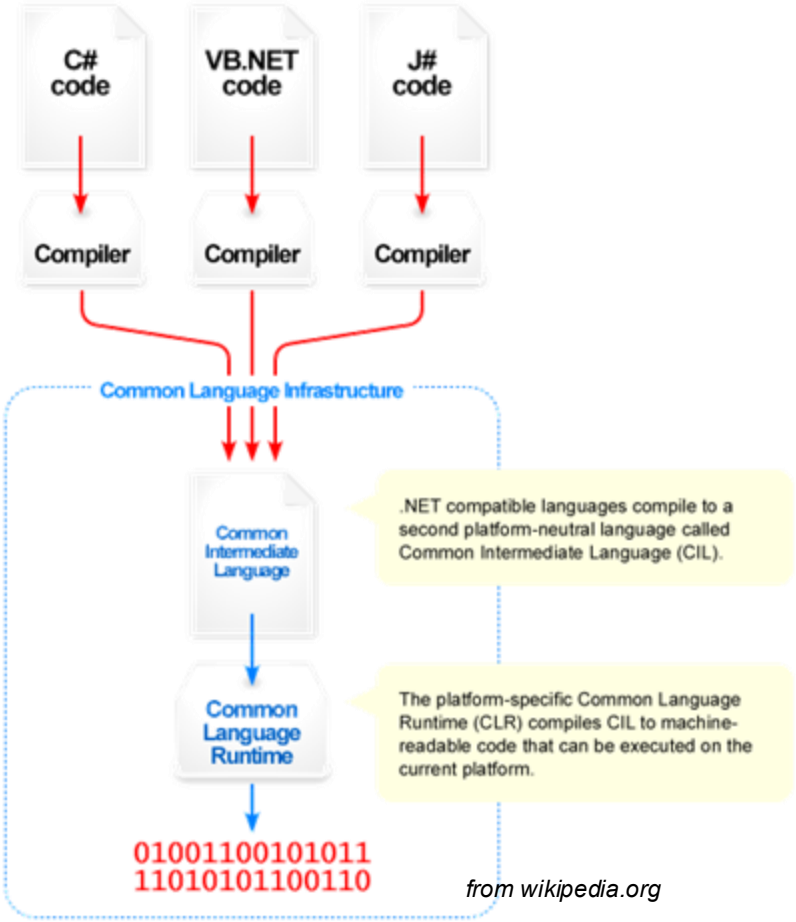
- C#, like Java, is executed indirectly through an abstract computer architecture called the CLR.
  - CLR => Common Language Runtime.
  - Abstract, but well defined.
- C# programs are compiled to an IL.
  - Also called MSIL, CIL (Common Intermediate Language) or bytecode.



- The CLR transforms the CIL to assembly instructions for a particular hardware architecture.
  - This is termed jit'ing or Just-in-time compiling.

CLR versus CLI

- CLR is actually an **implementation** by Microsoft of the CLI (Common Language Infrastructure) .
- CLI is an open *specification*.
- CLR is really a platform specific implementation.



from wikipedia.org

Future of C#

With C#’s flexibility and support for many languages through the .NET architecture it will definitely become a widely used language in all aspects of programming.